ResQCore OS Monitor: Feature and Demonstration Guide

Purpose and scope

The OS Monitor is an **isolated, seeded operating-system simulation** inside the ResQCore Command Centre. It models the command desk as one CPU and each demo SOS as a process (PCB). It demonstrates scheduling, a bounded intake queue, team mutual exclusion, and safe resource allocation.

The monitor is deliberately separate from live emergency operations:

- It owns an in-memory simulator created by `createMonitor()` in `backend/osMonitor.js`.
- Its seeded and scenario SOS records use demo IDs such as `DEMO-1001` and `DEMO-SURGE-01`.
- It does not process citizen SOS into real incidents or modify the live incidents, teams, or PostgreSQL assignment records.
- Its screens, actions, and API are for demonstrating the OS concepts. The live incident-verification and team-assignment routes are separate backend routes.

Open **Command Centre → OS Monitor**. The page is implemented in `CC/src/pages/OSMonitor.jsx`, with its presentation in `CC/src/pages/OSMonitor.css`. It shows the queue, PCBs, command-desk status, resource pool, Banker's table and safe sequence, and event log.

At a glance: the four pieces

Piece	OS concept	What to point to in the app
1. Process and scheduler	PCB, priority scheduling, preemption, aging, FCFS comparison	**Ready queue**, **Selected for operator review**, **Process control blocks**, and average-wait cards
2. Bounded intake queue	Producer–consumer and finite queue capacity	**Bounded intake** count and fill bar; **Surge · 15 SOS** scenario
3. Team lock	Mutual exclusion when assigning a team	**Verify & dispatch**, team availability/assignment, and the lock-denial behavior
4. Banker's algorithm	Maximum need, allocations, remaining need, safe-state check	**Banker's safety check**, resource totals, Need table, safe sequence, and **Unsafe request** scenario

The bottom-right **OS event log** records notable simulator events, including SOS arrival, preemption, aging, lock denial, allocation denial, dispatch, and completion. REST changes are also broadcast as `OS_MONITOR_UPDATED` WebSocket messages.

---

1. Process and scheduler

What the feature means

Every demo SOS is represented by a **process control block (PCB)**. In this model:

- **PID** identifies the demo process (for example, `DEMO-1001`).
- **State** indicates where it is in the command-desk lifecycle.
- **Arrival time** records when it entered the simulator.
- **Priority** and **tier** express urgency.
- **Wait time** is calculated from its arrival time while it is waiting.
- Its maximum resource need, current allocation, and request are also carried with the PCB for Banker's checks.

The command desk can handle only one running process. The scheduler orders waiting `READY` processes by effective priority, with earlier arrival time used to break a tie. The highest-ranked process is selected for operator review. A more urgent SOS may preempt that **review selection**. Once dispatch has made a process `RUNNING`, a new SOS does **not** interrupt that running process.

How priority is calculated

The simulator uses a simple score:

```
base score = severity score + category bonus + affected-people score
affected-people score = min(15, floor(people affected / 10))
final base score = min(100, base score)
```

Input	Scores used in the simulator
Severity	Critical 100, High 75, Medium 50, Low/Minor 25, Info 20
Category bonus	Fire +10, Medical +8, Rescue +8, Flood +5; other categories +0
People affected	+1 per 10 people, capped at +15

The three tiers are:

- **Immediate:** score of 80 or more.
- **Delayed:** score from 50 through 79.
- **Minor:** score below 50.

Example: a High-severity fire affecting 20 people scores `75 + 10 + 2 = 87`, so its tier is **Immediate**. A Medium-severity rescue affecting 12 people scores `50 + 8 + 1 = 59`, so its tier is **Delayed**.

Aging and preemption

While a PCB remains `READY`, its effective score rises by **20 points for each complete five minutes of waiting**, up to 100. Its wait time and tier are recalculated when the simulator produces a state snapshot or processes an enqueue operation. When its tier advances, an `AGED` event is added to the event log.

Example: a Low-severity request with no category bonus and one affected person starts at 25 (**Minor**). After ten minutes it receives two aging boosts: `25 + 20 + 20 = 65`, moving it to **Delayed**. Its increased score can move it ahead of another waiting PCB.

When a more urgent request arrives and becomes the highest-ranked PCB, the previously selected review candidate is put back among the READY processes and a `PREEMPTED` event is logged. This is **preemption before dispatch**—it is not a context switch between rescue teams and does not interrupt a running process.

FCFS versus Priority comparison

The monitor displays average waiting-time estimates for the same demo process list under:

1. **FCFS:** arrival-time order.
2. **Priority:** descending effective score, with arrival time as the tie-breaker.

For this comparison, the simulator uses tier-based synthetic service durations of 1 minute for Immediate, 2 minutes for Delayed, and 5 minutes for Minor. It accumulates the time each process waits before its turn, then averages those waits. This is a **small illustrative simulation**, not a measurement of actual emergency response times.

How to show it

1. Open **OS Monitor** and point to the **Ready queue** and **Process control blocks** table.
2. Point out each row's PID, state, tier/score, arrival time, wait, maximum need, and current need.
3. Click **Critical arrives**. The critical demo PCB should appear at the head of the ready queue, with a `PREEMPTED` event explaining that the previous review selection was paused.
4. Show the changed wait/tier or the `AGED` event for a process whose waiting time has crossed an aging interval. Aging is based on elapsed time; a fresh reset starts the seeded PCBs with staggered arrival times, but the UI does not provide a separate "advance clock" control.
5. Point to the **Avg. wait · FCFS** and **Avg. wait · Priority** cards for the simulated comparison.

Presentation wording: "Each SOS becomes a PCB. We calculate a priority score from its severity, category, and number of affected people. The READY queue is sorted by that score; waiting PCBs age upward. A critical SOS can take the operator's review slot from a less urgent request, but it does not interrupt a process already running at the command desk."

Implementation references

- `backend/osMonitor.js`: score and tiers in `getBasePriority()` / `tierForScore()`; aging in `priorityFor()` / `refreshAging()`; ordering in `sortReadyQueue()`; review preemption in `enqueue()`; comparison in `compareSchedulers()`.
- `CC/src/pages/OSMonitor.jsx`: ready queue, review card, PCB table, and FCFS/Priority metric cards.
- `backend/osMonitor.test.js`: aging promotion, priority wait comparison, and critical-review preemption tests.

---

2. Bounded intake queue (producer–consumer)

What the feature means

The simulator's intake capacity is **10 READY SOS processes**. Citizens are represented as producers: the **Surge · 15 SOS** demo generates 15 synthetic SOS arrivals. The command desk is the consumer: the operator reviews and dispatches the highest-ranked READY PCB.

The queue applies two overflow policies:

- If the queue is full and an incoming SOS is **Critical** and more urgent than the lowest-ranked queued request, it replaces that lowest-priority READY request. The queue stays at capacity and the event log records `QUEUE_REPLACED`.
- A non-qualifying incoming SOS is rejected with a **"queue full; retry later"** message and a `QUEUE_FULL` event. This includes a low-priority SOS at capacity.

The queue count tracks READY processes. A process dispatched to RUNNING is no longer counted as waiting in the READY intake queue.

How to show it

1. Click **Surge · 15 SOS**.
2. Point to **Bounded intake** and its fill bar. The surge generates 15 demo arrivals, but the displayed queue never exceeds **10**.
3. Point to the event log for `QUEUE_REPLACED` and `QUEUE_FULL`. The surge's deterministic severity sequence includes Critical arrivals and lower-priority arrivals that are rejected.
4. The surge button summarizes how many demo SOS were processed and how many remain in the queue.
5. Click **Reset demo** to restore the seeded starting data.

Presentation wording: "The surge button acts as a burst of citizen producers. The intake queue has a fixed capacity of ten. At capacity, a sufficiently urgent Critical SOS can displace the least urgent queued item; a lower-priority request gets a retry response instead of silently overflowing the queue."

Implementation references

- `backend/osMonitor.js`: `QUEUE_CAPACITY`, `enqueue()`, and the `surge` case in `runScenario()`.
- `backend/server.js`: `POST /api/os-monitor/incidents` returns `201` when accepted and `429` when the queue rejects the request; the scenario route runs the seeded surge.
- `CC/src/pages/OSMonitor.jsx`: Surge button and queue count/fill bar.
- `backend/osMonitor.test.js`: capacity/replacement, low-priority retry, and surge-bounds tests.

---

3. Team lock (mutual exclusion)

What the feature means

A rescue team is a shared resource: two operators must not assign the same available team at the same time. Before a team is assigned, the backend acquires a lock for that team. If the team is already locked or is no longer available, the request is rejected with an explicit **"team locked"** message and (in the OS simulator) a `LOCK_DENIED` event. The lock is released after the assignment attempt, including when that attempt fails.

For the OS Monitor, the two seeded teams are `DEMO-TEAM-1` and `DEMO-TEAM-2`. A successfully dispatched team changes to **Deployed** and is linked to the RUNNING PCB. Completing that PCB makes the team available again.

There is also live-operation protection separate from the demo: `backend/server.js` uses a server-side `Set` lock around the live verify/assign handlers. PostgreSQL's partial unique index provides a persistent database-level backup that permits only one unreleased assignment per team.

How to show it

1. Click **Reset demo**. The simulator state starts with two available demo teams (`DEMO-TEAM-1` and `DEMO-TEAM-2`); the screen offers an available team when dispatching.
2. Click **Verify & dispatch** for the selected PCB. The success message identifies the chosen team; that team is marked **Deployed** in simulator state while the PCB becomes RUNNING.
3. Explain that another assignment cannot take that same team while it is locked or deployed; the server rejects the competing attempt with a "team locked" message.
4. Click **Mark done & release resources**. The simulator releases the assignment and the team becomes **Available** again.

Important demo limitation: the monitor UI has one operator-review button and disables its controls while a request is in flight, so the screen does not provide a dedicated two-operator race-demo button. The lock-denial path is implemented and covered by a unit test that simulates the team already being locked; the live backend routes also enforce the lock. Do not describe this as a visible two-user concurrency demo unless one is added.

Presentation wording: "The team is protected by mutual exclusion. The backend takes a team-specific lock before assignment and releases it after the operation. If another assignment finds the team locked or deployed, it receives a conflict response. The database's unique index is a second line of defense against duplicate active assignments."

Implementation references

- `backend/osMonitor.js` : `dispatch()` checks availability, acquires `state.teamLocks` , and releases it in `finally` ; `complete()` restores the team to Available.
- `backend/server.js` : `acquireTeamLock()` and the live `/api/incidents/:id/verify` and `/api/incidents/:id/assign` handlers.
- `backend/database.js` : `one_active_assignment_per_team` partial unique index on active (unreleased) assignments.
- `backend/osMonitor.test.js` : explicit lock-denial test and dispatch/completion test.

4. Banker's algorithm

What the feature means

Instead of using a single supply count for the OS simulation, each PCB declares its **maximum** possible resource need. The simulator tracks four resource types:

Resource	Seeded total
Ambulances	3
Boats	2
Medical kits	10
Food packs	24

For every process, the Banker's table displays:

```
Need = Maximum − Allocation
```

Before dispatch, the simulator validates the request:

1. The request cannot exceed the PCB's remaining maximum need.
2. The request must fit within the currently available pool.
3. It tentatively applies the allocation and runs the safety check.
4. It grants the request only if all processes can still finish in some order—the **safe sequence**. Otherwise it rolls back by not committing the tentative allocation and explains which processes are blocked.

On successful dispatch, the requested resources are reserved and the available totals decrease. On completion, those resources are released back to the pool.

Unsafe request example

Click **Unsafe request**. The simulator loads a separate three-process Banker example with totals of 10 ambulances, 5 boats, 7 medical kits, and 6 food packs, plus existing allocations. It attempts to give `DEMO-OS-P0` **5 additional ambulances**. Although the request is within P0's remaining declared maximum and there are initially 5 ambulances available, granting it leaves no safe completion sequence. The request is denied; the monitor shows **DENIED**, the reason, the resource table, and the original safe sequence.

This scenario deliberately changes the **demo Banker's state only**. It is not a request for real rescue resources. Reset the demo before trying dispatch again.

Safe dispatch and release example

In the seeded state, `DEMO-1001` requests 1 ambulance and 2 medical kits. The simulator checks that this fits the PCB's maximum and available pool and that a safe sequence remains. If safe, it grants those quantities, marks the PCB RUNNING, and assigns the demo team. **Mark done & release resources** then returns the allocation, terminates the PCB, frees the command desk, and makes the team available.

How to show it

1. Point to **Banker's safety check**: total/available cards, each process's Maximum, Allocation, and Need, and the **Safe sequence**.
2. Click **Unsafe request**. Show the DENIED badge and the "no safe sequence" explanation.
3. Click **Reset demo**. Click **Verify & dispatch** for the selected seeded PCB and show the GRANTED result and reduced available counts.
4. Click **Mark done & release resources** and show the resource counts return to their previous values.

Presentation wording: "Before committing a resource request, the Banker checks the process's declared maximum and the available pool, then simulates the allocation. It grants only if a safe order still exists in which all processes can finish. When the process completes, its allocation returns to the pool."

Implementation references

- `backend/osMonitor.js`: resource vectors and totals; `getNeed()`; `findSafeSequence()`; `requestIsSafe()`; `dispatch()`; `complete()`; and the `unsafe` case in `runScenario()`.
- `CC/src/pages/OSMonitor.jsx`: resource cards, Banker's table, safe sequence, last request result, and dispatch/completion controls.
- `backend/osMonitor.test.js`: unsafe-request denial and safe dispatch/resource-release tests.

End-to-end demonstration script

Use this sequence when asked to explain how the pieces work together:

1. Open **Command Centre → OS Monitor** and identify that the screen is marked **DEMO DATA ONLY**.
2. Point out the seeded READY PCBs and the scheduler-selected process. Explain the score, tiers, wait time, and synthetic FCFS/Priority comparison.
3. Click **Surge · 15 SOS**. Show the bounded queue at no more than 10 and the overflow/replacement events.
4. Reset, then click **Critical arrives**. Show the Critical PCB at the top and the `PREEMPTED` log entry for the prior review candidate.
5. Reset, then click **Unsafe request**. Show Banker's denial and its unsafe-state explanation.
6. Reset, then click **Verify & dispatch**. Show the PCB move to RUNNING, the team become deployed, resources be granted, and the safe-sequence display.
7. Click **Mark done & release resources**. Show the PCB become TERMINATED, the resource pool recover, and the team become available.
8. Remind the audience that all SOS and resources in this page are simulated demo data, not real incident dispatches.

API and realtime path

The Command Centre calls these simulator routes in `backend/server.js`:

Method and path	Purpose
`GET /api/os-monitor`	Read the current simulator snapshot
`POST /api/os-monitor/reset`	Restore the seeded simulator
`POST /api/os-monitor/scenarios/surge`	Run the 15-SOS surge
`POST /api/os-monitor/scenarios/critical`	Enqueue a critical demo SOS
`POST /api/os-monitor/scenarios/unsafe`	Demonstrate an unsafe Banker request
`POST /api/os-monitor/incidents`	Add a validated demo SOS to the bounded queue
`POST /api/os-monitor/dispatch/:pid`	Attempt a team assignment and safe resource grant
`POST /api/os-monitor/complete/:pid`	Terminate the running demo PCB and release resources

Each successful simulator mutation broadcasts `OS_MONITOR_UPDATED` over the existing `/ws` WebSocket. The monitor also refreshes its snapshot periodically, so REST responses and the event stream both update the display.

Scope and implementation notes

These distinctions help avoid overstating what the current code does:

- New simulated PCBs are created directly in **READY**. The PCB has a state field and the monitor demonstrates `READY`, `RUNNING`, and `TERMINATED`; **NEW is not currently a separately displayed or persisted transition**.
- The intended scheduling preemption is of the selected **review candidate** while it is still READY. A RUNNING PCB is not preempted.
- Aging is calculated from elapsed wall-clock time when the snapshot/enqueue logic refreshes the state; there is no manual "advance time" control on the screen.
- The OS Monitor uses four typed resource pools for the simulator. The existing live dashboard's `reliefSupplies` statistic remains separate; it is not converted into Banker's resource accounting by this demo.
- Team-lock conflict handling is implemented and unit-tested, but the monitor does not include a dedicated two-operator concurrency button.

Test coverage

Run the backend OS Monitor tests from the `backend` directory:

```
npm test
```

`backend/osMonitor.test.js` covers aging promotion, bounded capacity and critical replacement, low-priority retry at capacity, Banker's unsafe-request denial, the FCFS/Priority wait comparison, seeded surge isolation, review-candidate preemption, safe dispatch and completion/resource release, command-desk occupancy, and explicit team-lock denial.

Code map

The following locations make it easy to open the implementation while presenting:

File	Source locations	Responsibility
`backend/osMonitor.js`	Lines 1–24	Queue/aging constants, resource names, severity scores, category bonuses
`backend/osMonitor.js`	Lines 58–106	Priority score, three tiers, aging score, PCB construction and seeded PCBs
`backend/osMonitor.js`	Lines 203–258	Banker's safe-sequence and tentative request checks
`backend/osMonitor.js`	Lines 275–304	READY-queue ordering and aging event generation
`backend/osMonitor.js`	Lines 333–355	FCFS-versus-priority average-wait model
`backend/osMonitor.js`	Lines 384–426	Queue capacity handling, replacement/retry, and review preemption
`backend/osMonitor.js`	Lines 478–543	Surge, critical-arrival, unsafe-request, and reset scenarios
`backend/osMonitor.js`	Lines 545–609	Team lock, resource grant, RUNNING state, termination and release
`backend/server.js`	Lines 25–30, 103–155	Shared demo state and OS Monitor REST endpoints
`backend/server.js`	Lines 263–321 and 326–384	Live verify/assign lock paths, separate from the seeded OS simulation
`backend/database.js`	Line 71	PostgreSQL unique index for one active assignment per team
`CC/src/pages/OSMonitor.jsx`	Lines 114–176	Demo controls, queue fill bar, command desk and average-wait cards
`CC/src/pages/OSMonitor.jsx`	Lines 184–258	Ready queue, review/dispatch controls and PCB table
`CC/src/pages/OSMonitor.jsx`	Lines 266–326	Banker's table, safe sequence, resource result and OS event log
`backend/osMonitor.test.js`	Lines 14–140	Nine focused tests for the behaviors described above